

SLIIT
SE3082 – Parallel Computing



REPORT
LAB 05

Akash D
IT24101853

Exercise 06

Code Implementation

```
GNU nano 9.1 lab05f.c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 1000000
#define STRIP_SIZE 256 // Strip size aligned with cache/vector characteristics

int main() {
    double *A = (double *)malloc(N * sizeof(double));
    double *B = (double *)malloc(N * sizeof(double));
    double *C = (double *)malloc(N * sizeof(double));

    // Initialize arrays A and B
    for (int i = 0; i < N; i++) {
        A[i] = 1.5;
        B[i] = 2.0;
    }

    double tstart = omp_get_wtime();

    // Strip Mining Implementation with OpenMP
    #pragma omp parallel
    {
        int total_threads = omp_get_num_threads();
        int tid = omp_get_thread_num();

        // Outer loop for strip mining: divide the total range into strips of size STRIP_SIZE
        #pragma omp for schedule(static)
        for (int s = 0; s < N; s += STRIP_SIZE) {
            int strip_end = s + STRIP_SIZE;
            if (strip_end > N) {
                strip_end = N;
            }

            // Inner loop processing the elements within the current strip
            for (int i = s; i < strip_end; i++) {
                C[i] = A[i] * B[i];
            }
        }
    }

    double tstop = omp_get_wtime();

    // Verify a sample of the results
    printf("Sample Results:\n");
    printf("C[0] = %.2f, C[500000] = %.2f, C[999999] = %.2f\n", C[0], C[500000], C[999999]);
    printf("Execution Time with Strip Mining and OpenMP: %.4f ms\n", (tstop - tstart) * 1000.0);

    // Free allocated memory
    free(A);
    free(B);
    free(C);

    return 0;
}
```

Output

```
/a/3 Semester/PC/Lab05/Exercise06
LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise06
$ gcc -fopenmp lab05f.c -o lab05f.o

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise06
$ ./lab05f.o
Sample Results:
C[0] = 3.00, C[500000] = 3.00, C[999999] = 3.00
Execution Time with Strip Mining and OpenMP: 0.0000 ms

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise06
$ |
```

Strip Mining with OpenMP

- What is Strip Mining? Strip mining is a loop transformation technique that breaks a large iteration space ($N = 1,000,000$) into smaller, manageable chunks or "strips" of a fixed size ($STRIP_SIZE = 256$).
- Cache and SIMD Optimization: By keeping strips small and localized, the working data fits efficiently into processor cache lines and hardware vector registers, maximizing spatial locality and enabling auto-vectorization by the compiler.
- OpenMP Scheduling: The outer strip loop uses `#pragma omp parallel for schedule(static)` to distribute blocks of strips across multiple threads evenly with minimal scheduling overhead.